

中国科学院大学

University of Chinese Academy of Sciences

Generative Models & AI4S Applications

Lecture 4 Maximum Likelihood

Fall 2026

Liu Yunfan (刘云帆)

liuyunfan@ucas.ac.cn

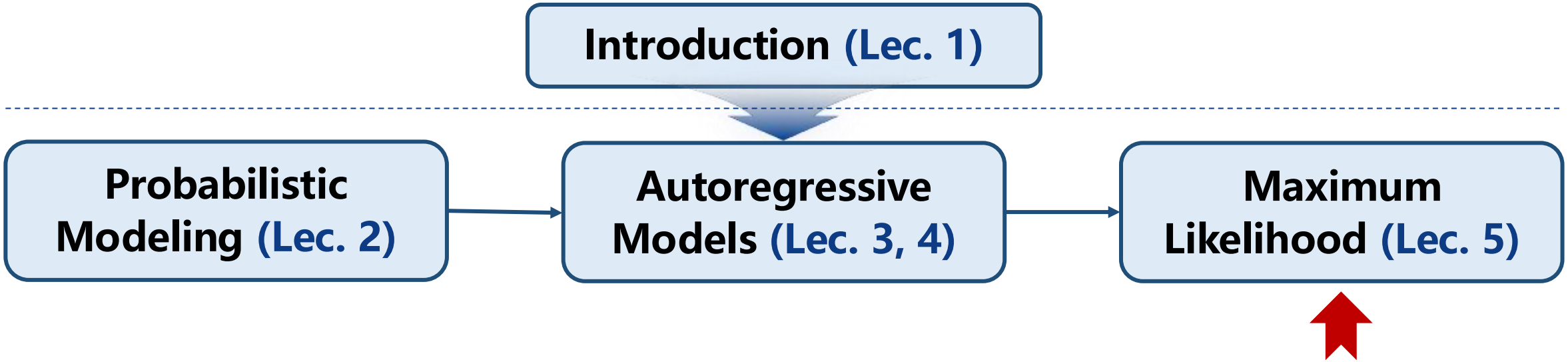
Announcements

- Assignment 1 has released online and is **due on October 18th**

- For questions, please contact the TAs by email or WeChat group

- **Late Submission Policy**
  - Late submissions are accepted **within one week** after the deadline
  - The maximum score for late submissions is **80%**
  - Submissions more than one week late will receive **no credit**
  - Please send late submissions directly to the TAs by email

Roadmap



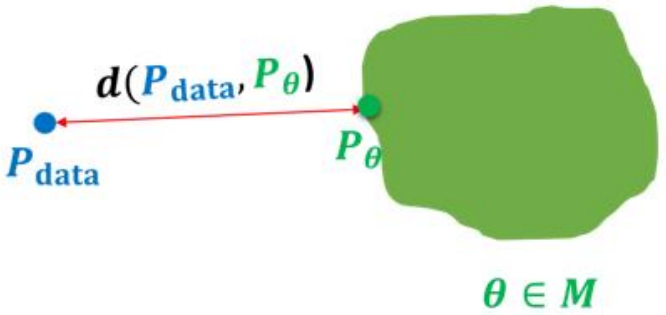
Specifying the objective function for learning the model p_θ

Principles of Statistical Machine Learning

Images of dogs



$\mathbf{x}_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



Dog Image Generator

- Three main components:
  - **Model Family (模型):** Define the scope of learnable function (i.e., $P_\theta(x)$)
  - **Learning Objective (准则):** Quantify the target of learning process $\mathcal{L}(x; \theta)$
  - **Optimization (算法):** Algorithm for solving the optimal parameters θ^*

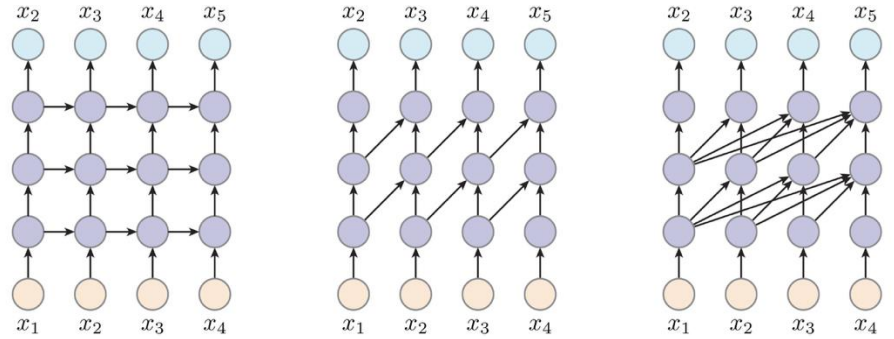
$$\theta^* = \arg\min_\theta \mathbb{E}_{x \sim p_{\text{data}}(x)}[\mathcal{L}(x; \theta)]$$

Recap

- **Introduction to Autoregressive Models**
  - Flexible, easy to sample from, **exact likelihood evaluation**
  - **Ordering** is the key of AR modeling (Inductive Bias)
- **Basic Autoregressive Models**

$$p(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, x_2, \ldots, \boldsymbol{x_{<i}})$$

  - FVSBN, NADE, MADE
- **Network Architectures for Autoregressive Modeling**
  - RNN, CNN, Attention
- **Inference and Training Scheme**
  - Teacher Forcing

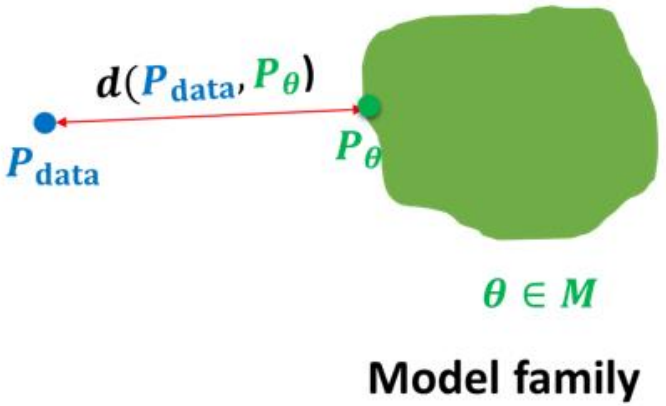


Principles of Statistical Machine Learning

Images of dogs



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



Dog Image Generator

- Three main components:
  - **Model Family (模型):** Define the scope of learnable function (i.e., $P_\theta(x)$)
  - **Learning Objective (准则):** Quantify the target of learning process $\mathcal{L}(x;\theta)$
  - **Optimization (算法):** Algorithm for solving the optimal parameters θ^*

$$\theta^* = \arg\min_\theta \mathbb{E}_{x \sim p_{\text{data}}(x)}[\mathcal{L}(x;\theta)]$$



Lecture Outline

- **Learning via Maximum Likelihood**
- **Maximum Likelihood from Data**
- **Statistical Issues and the Bias-Variance Tradeoff**

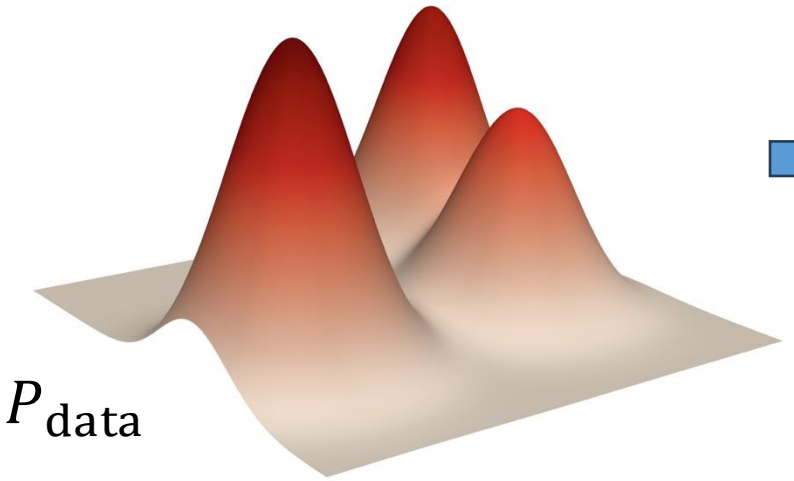


Lecture Outline

- **Learning via Maximum Likelihood**
- **Maximum Likelihood from Data**
- **Statistical Issues and the Bias-Variance Tradeoff**

Learning via Maximum Likelihood

- **Data Settings**
  - **Assumed** to come from an **underlying** distribution P_{data} $(x \sim P_{\text{data}})$
  - Given a dataset $\mathcal{D}$ consists of m samples from P_{data}
    - ✓ Each sample is an **assignment of values** to the variables x
    - ✓ Samples are **independently and identically distributed (i.i.d)**

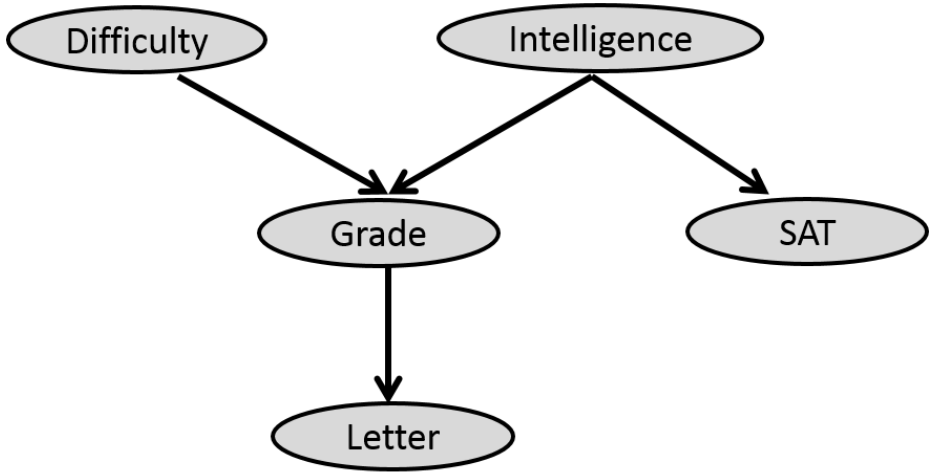


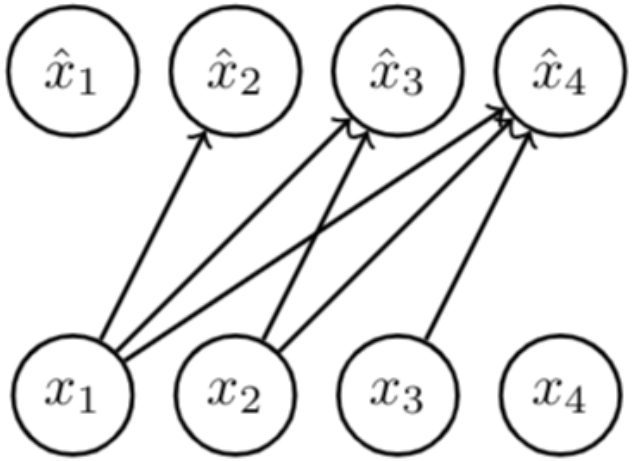


$\mathbf{x}_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$

Learning via Maximum Likelihood

- **Model Settings**
  - ➢ Learning is performed within a **family of models** $\mathcal{M}$
    - ✓ Given $G = (V, E)$, $\mathcal{M}$ is all Bayes nets with all possible CPT
    - ✓ Given a FVSBN, $\mathcal{M}$ is all possible choices of LR parameters $\theta \in \Theta$

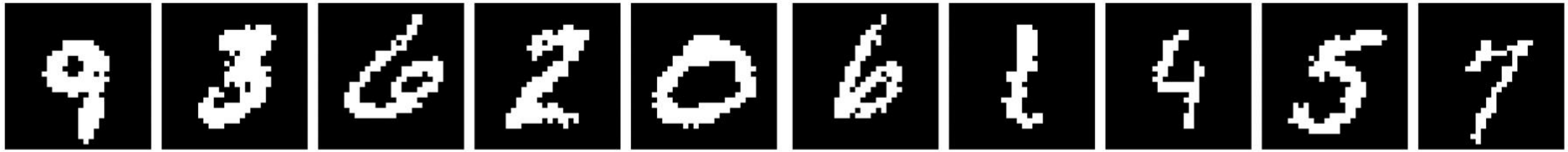




Model family $\mathcal{M}$ defined by computation graph

Learning via Maximum Likelihood

- **Goal of Learning**
  - **Goal:** Learn $P_\theta \in \mathcal{M}$ that **precisely captures** the distribution P_{data}
  - **Challenges:**
    - ✓ The true underlying distribution P_{data} is **unknown**
    - ✓ **Limited Data** $\mathcal{D}$ only provides a rough approximation of P_{data}



    - ✓ How many possible states?
    - ✓ How many samples available?

Learning via Maximum Likelihood

- **Goal of Learning**
  - ➢ **Goal:** Learn $P_\theta \in \mathcal{M}$ that **precisely captures** the distribution P_{data}

    ➔ Select P_θ to **best approximate** P_{data}
  - ➢ **Question:** What is "*best*" ?

    ➔ Depends on what we want to do:

    ✓ **Density Estimation:** Care about the full distribution, enables probability computation for arbitrary query inputs

    ✓ **Generation Quality:** Sample realistic x_{new} ("plausible")

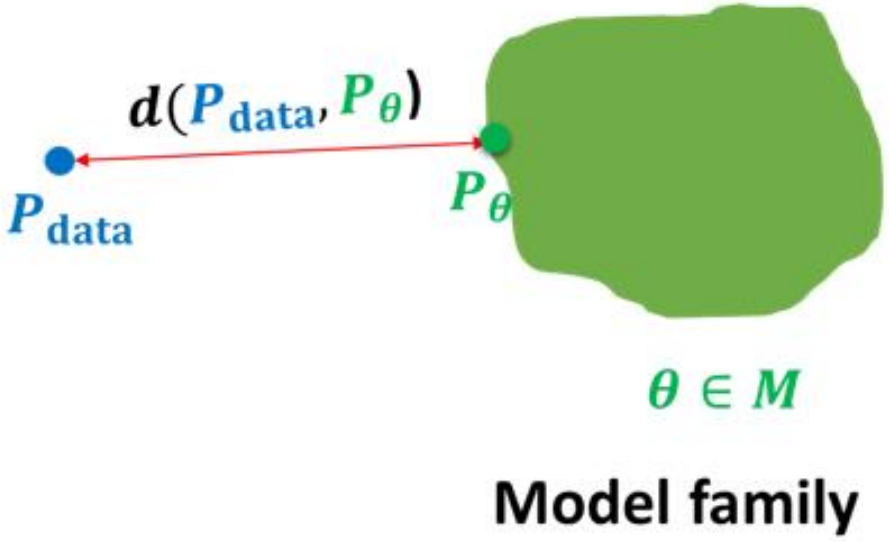
    ✓ **Structure Discovery:** Meaningful representation learning

Learning via Maximum Likelihood

- **Learning as Density Estimation**
  - **Goal:** Learn the **full distribution** $P_\theta(x)$, enabling computation of any query
  - **"Regression":** Construct P_θ as **"close"** as possible to P_{data}



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



  - **Goal:** Choose an objective $d(P_\theta, P_{\text{data}})$ to **capture the "closeness"**

Learning via Maximum Likelihood

- **How to Define "Closeness" between Distributions?**
  - **Kullback-Leibler (KL) divergence** between distributions p and q is

$$D(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

  - **Intuitive understandings:**
    ① The log-ratio is the **per-sample penalty** of using q instead of p

    ② Mis-matches on high-probability samples under p are penalized more (i.e., $p(x)$ as a **importance weighting** term)

    ③ Information theory explanation: $D(p \parallel q) = H(p) - H(p, q)$

Learning via Maximum Likelihood

- **How to Define "Closeness" between Distributions?**
  - **Kullback-Leibler (KL) divergence** between distributions p and q is

$$D(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

➔ $D(p \parallel q) \geq 0$ for all p and q ($D = 0$ iff $p = q$)

$$D(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \sum_x p(x) \left[-\log \frac{q(x)}{p(x)} \right] = \mathbb{E}_{x \sim p} \left[-\log \frac{q(x)}{p(x)} \right]$$

$$\Rightarrow \mathbb{E}_{x \sim p} \left[-\log \frac{q(x)}{p(x)} \right] \geq -\log \left(\mathbb{E}_{x \sim p} \left[\frac{q(x)}{p(x)} \right] \right) = -\log \left(\sum_x p(x) \frac{q(x)}{p(x)} \right) = 0$$

Learning via Maximum Likelihood

- **How to Define "Closeness" between Distributions?**
  - **Kullback-Leibler (KL) divergence** between distributions p and q is

$$D(p \,\|\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

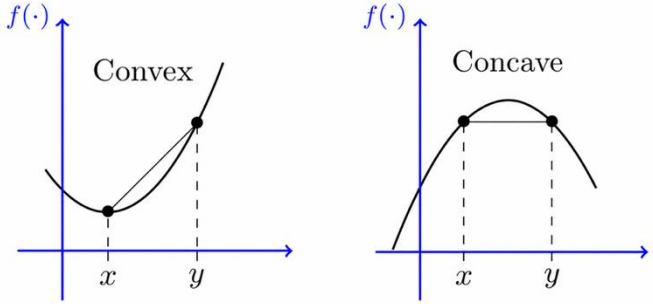
➔ $D(p \,\|\, q) \geq 0$ for all p and q ($D = 0$ iff $p = q$)

$$\mathbb{E}_{x\sim p}\left[-\log \frac{q(x)}{p(x)}\right] \geq -\log\left(\mathbb{E}_{x\sim p}\left[\frac{q(x)}{p(x)}\right]\right) = -\log\left(\sum_x p(x)\frac{q(x)}{p(x)}\right) = 0$$

Jensen's Inequality

$$\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$$

for any convex function f

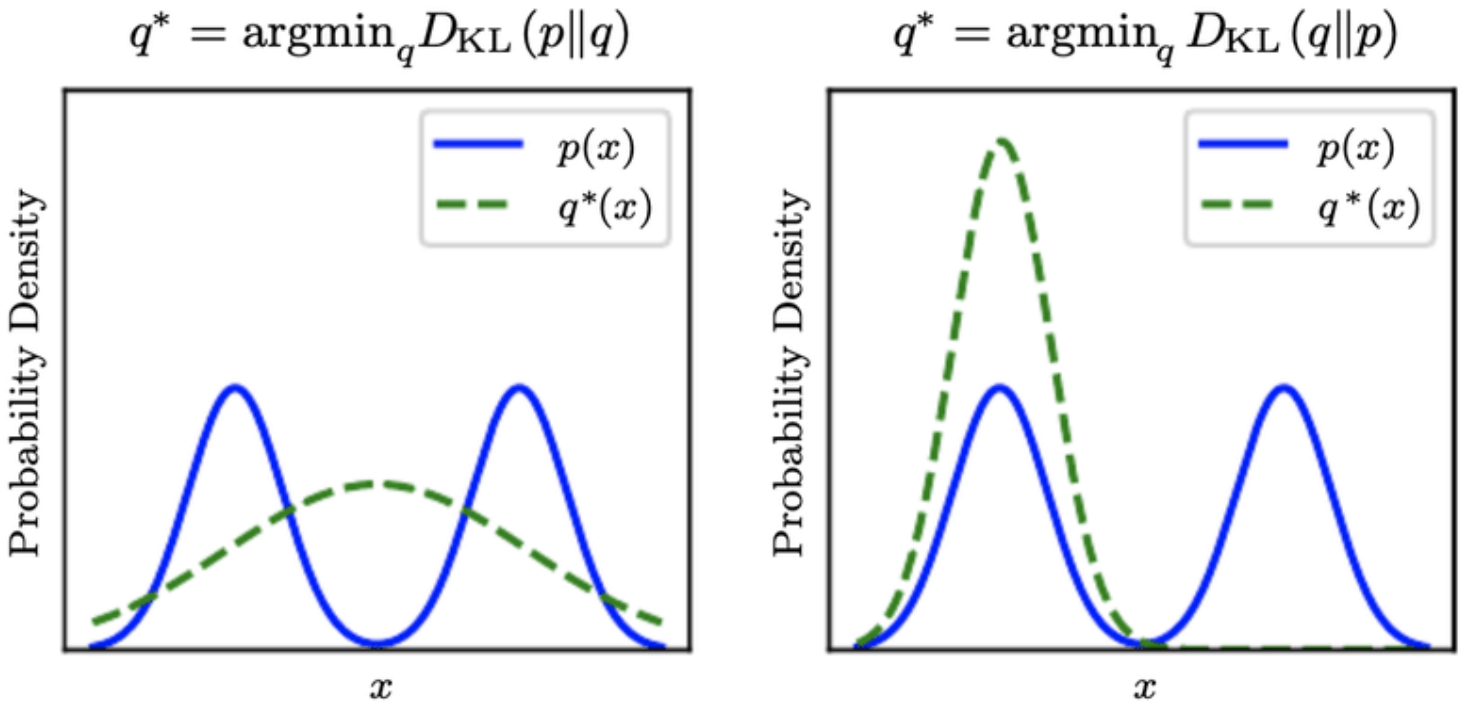


Learning via Maximum Likelihood

- **Kullback-Leibler (KL) Divergence**

$$D(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

  - KL divergence is used to quantify the "similarity" of p and q
  - **Note:** KL divergence is **asymmetric**, i.e., $D(p \parallel q) \neq D(q \parallel p)$



mode-covering **mode-seeking**

Learning via Maximum Likelihood

- **KL Divergence as Objective**
  - ➢ We use KL divergence to estimate the "similarity" between P_{data} and P_θ

$$\mathbf{D}(P_{\text{data}}||P_\theta) = \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log\left(\frac{P_{\text{data}}(\mathbf{x})}{P_\theta(\mathbf{x})}\right)\right] = \sum_{\mathbf{x}} P_{\text{data}}(\mathbf{x}) \log \frac{P_{\text{data}}(\mathbf{x})}{P_\theta(\mathbf{x})}$$

  - ➔ $D(P_{\text{data}} \parallel P_\theta) = 0$ iff $P_{data} = P_\theta$
  - ➢ $D(P_{\text{data}} \parallel P_\theta)$ can be simplified as

$$\begin{aligned}\mathbf{D}(P_{\text{data}}||P_\theta) &= \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log\left(\frac{P_{\text{data}}(\mathbf{x})}{P_\theta(\mathbf{x})}\right)\right] \\ &= \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_{\text{data}}(\mathbf{x})\right] - \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_\theta(\mathbf{x})\right]\end{aligned}$$

Learning via Maximum Likelihood

- **KL Divergence as Objective**

$$\begin{aligned}\mathbf{D}(P_{\text{data}}||P_\theta) &= \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log\left(\frac{P_{\text{data}}(\mathbf{x})}{P_\theta(\mathbf{x})}\right)\right] \\ &= \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_{\text{data}}(\mathbf{x})\right] - \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_\theta(\mathbf{x})\right]\end{aligned}$$

➢ **Minimizing KL divergence ⟺ Maximizing the expected log-likelihood**

$$\arg\min_{P_\theta} \mathbf{D}(P_{\text{data}}||P_\theta) = \arg\min_{P_\theta} -\mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_\theta(\mathbf{x})\right] = \arg\max_{P_\theta} \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_\theta(\mathbf{x})\right]$$

✓ P_θ assign high probability (“likelihood”) to data sampled from P_{data}

✓ If $P_\theta \approx 0$ for $x \sim P_{\text{data}}$, then penalty $\log(P_\theta) \to -\infty$ (“mode-covering”)

Learning via Maximum Likelihood

- **KL Divergence as Objective**

$$\mathbf{D}(P_{\text{data}}||P_\theta) = \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log\left(\frac{P_{\text{data}}(\mathbf{x})}{P_\theta(\mathbf{x})}\right)\right]$$

$$= \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_{\text{data}}(\mathbf{x})\right] - \mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_\theta(\mathbf{x})\right]$$

Negative log-Likelihood (NLL)

$$\mathbf{H}(P_{\text{data}}) = -\mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_{\text{data}}(\mathbf{x})\right]$$

- Although we can now compare models and reduce the KL divergence, we still do not know how close we are to the optimum
  - ✓ **Fundamental problem:** we do not know P_{data}



Lecture Outline

- **Learning via Maximum Likelihood**
- **Maximum Likelihood from Data**
- **Statistical Issues and the Bias-Variance Tradeoff**

Maximizing the Likelihood from Data

- **Monte Carlo Estimation**
  - Quantity of interest ➔ Expected value of a random variable

$$E_{x\sim P}[g(x)] = \sum_{x} g(x)P(x)$$

  - **Sampling**: Generate T samples $x^1, \ldots, x^T$ from the distribution $P(x)$, i.e., where the expectation was take
  - **Estimation**: Approach the expected value from the samples

$$E_{x\sim P}[g(x)] \Rightarrow \hat{g}(\mathbf{x}^1, \cdots, \mathbf{x}^T) \triangleq \frac{1}{T}\sum_{t=1}^{T} g(\mathbf{x}^t)$$

MC Estimation: Approximate expectations with sample averages

Maximizing the Likelihood from Data

- **Properties of Monte Carlo Estimation** $E_{x\sim P}[g(x)] \Rightarrow \frac{1}{T}\sum_{t=1}^{T} g(\mathbf{x}^t)$
  - **Unbiased**

$$E_P[\hat{g}] = E_P[g(x)]$$

  - **Convergence:** By law of large numbers

$$\hat{g} = \frac{1}{T}\sum_{t=1}^{T} g(x^t) \rightarrow E_P[g(x)] \text{ for } T \rightarrow \infty$$

  - **Variance**: Can be reduced by increasing the number of samples (T)

$$V_P[\hat{g}] = V_P\left[\frac{1}{T}\sum_{t=1}^{T} g(x^t)\right] = \frac{V_P[g(x)]}{T}$$

Maximizing the Likelihood from Data

- **Monte Carlo Estimation for Maximizing the Likelihood**
  - **Motivation:** Approximate the expectation with empirical log-likelihood

$$\mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}\left[\log P_\theta(\mathbf{x})\right] \Rightarrow \mathbf{E}_{\mathcal{D}}\left[\log P_\theta(\mathbf{x})\right] = \frac{1}{|\mathcal{D}|}\sum_{\mathbf{x}\in\mathcal{D}} \log P_\theta(\mathbf{x})$$

  - **Maximum Likelihood Learning**

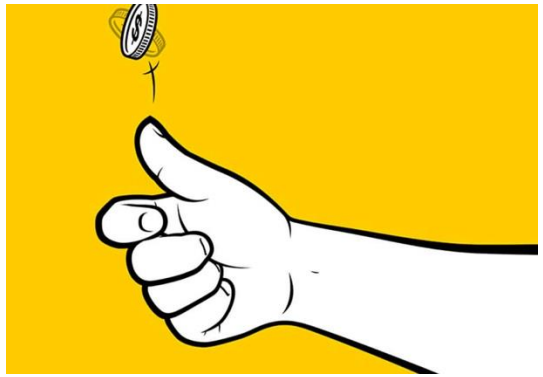
$$\max_{P_\theta} \frac{1}{|\mathcal{D}|}\sum_{\mathbf{x}\in\mathcal{D}} \log P_\theta(\mathbf{x})$$

This is equivalent to maximize the joint probability of (i.i.d.) data

$$P_\theta(\mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(m)}) = \prod_{\mathbf{x}\in\mathcal{D}} P_\theta(\mathbf{x})$$

Maximizing the Likelihood from Data

- **Maximizing Likelihood: Coin Flipping Example**
  - **Variable:** A scalar with two outcomes, i.e., $Heads\ (H)$ and $Tails\ (T)$
  - **Dataset:** Record of Tosses, e.g., $\mathcal{D} = \{H, H, T, H, T\}$



  - **Formalization:** The process is controlled by $P_{\text{data}}(x), x \in \{H, T\}$
  - **Model Family $\mathcal{M}$:** All Bernoulli distributions over $x \in \{H, T\}$
  - **Learning Problem:** How to choose $P_\theta(x)$ from $\mathcal{M}$ to "best resemble" $\mathcal{D}$?

Maximizing the Likelihood from Data

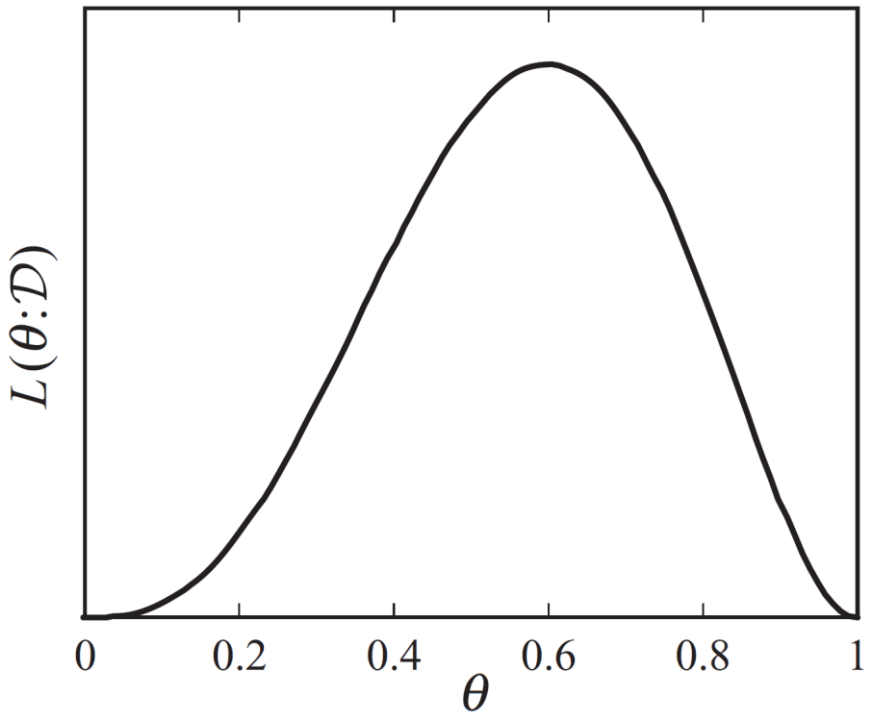
- **Maximizing Likelihood**
  - **Model Parameterization:**

$$P_\theta(x = H) = \theta \qquad P_\theta(x = T) = 1 - \theta$$

  - **Likelihood Maximization:** Optimize for θ which makes $\mathcal{D}$ most likely
    - ✓ **Data Likelihood:** Probability of observing $\mathcal{D}$ / Chance of sampling $\mathcal{D}$

$$\prod_i P_\theta(x_i) = \theta \cdot \theta \cdot (1-\theta) \cdot \theta \cdot (1-\theta)$$

$$\mathcal{D} = \{H, H, T, H, T\}$$



$$\prod_i P_\theta(x_i)$$

Maximizing the Likelihood from Data

- **Maximizing Likelihood: Coin Flipping Example**
  - **Distributions:** $P_\theta(x = H) = \theta \qquad P_\theta(x = T) = 1 - \theta$
  - **Likelihood for $\mathcal{D}$:**

$$\begin{aligned} L(\theta) &= \theta^{\#heads} \cdot (1-\theta)^{\#tails} \\ \log L(\theta) &= \log(\theta^{\#heads} \cdot (1-\theta)^{\#tails}) \\ &= \#heads \cdot \log(\theta) + \#tails \cdot \log(1-\theta) \end{aligned}$$

  - **Goal for MLE:** Find $\theta^* \in [0,1]$ such that $\log L(\theta^*)$ is maximum
  - **Solution:** Let $\nabla_\theta \log L(\theta) = 0$, we have

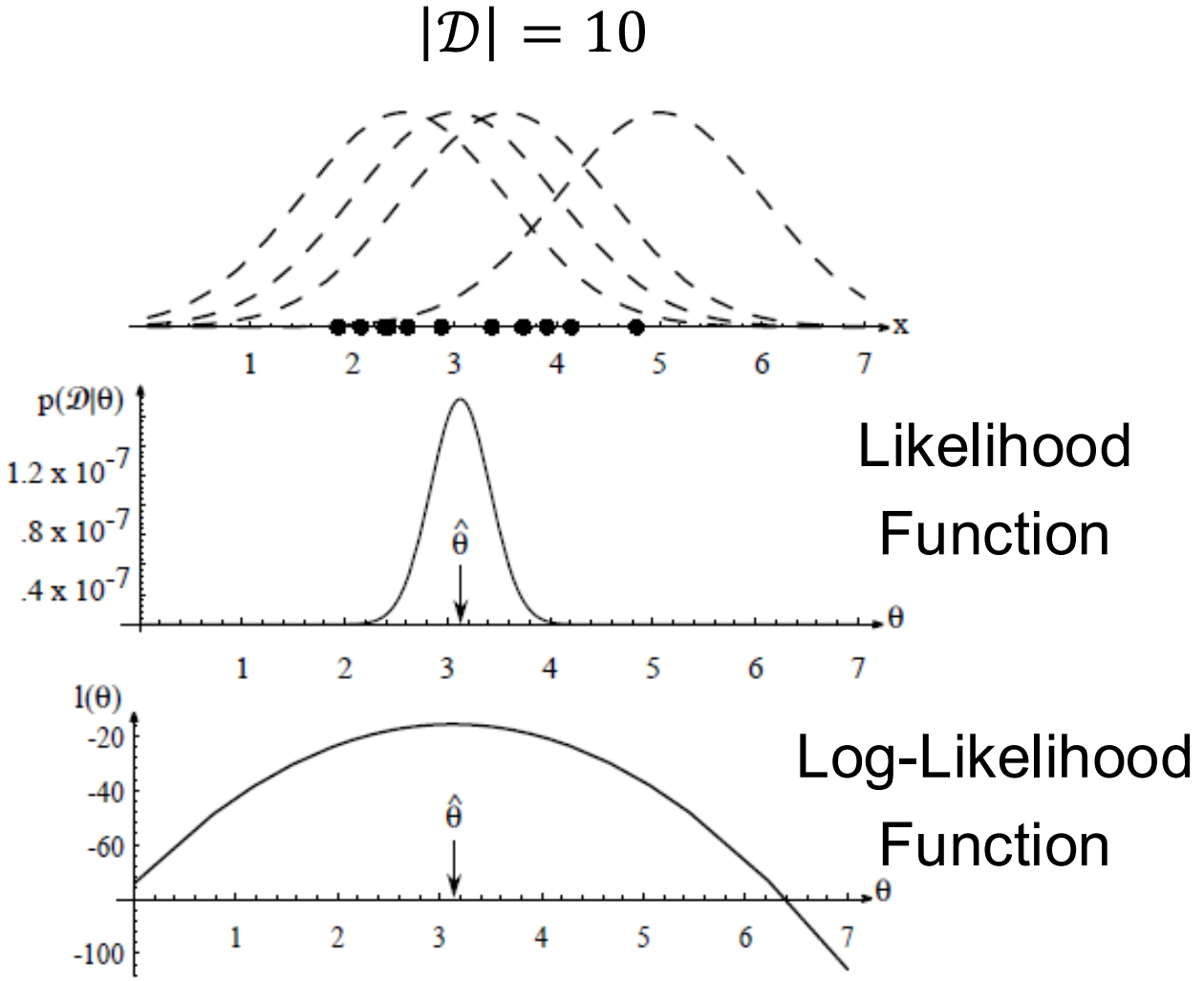
$$\theta^* = \frac{\#heads}{\#heads + \#tails}$$

Maximizing the Likelihood from Data

- **Maximizing Likelihood:**

  The Body Weight Example

  - $\mathcal{D}$ is drawn from $\mathcal{N}(\mu, \sigma^2)$
    - ✓ Mean value μ is unknown (variable θ)
    - ✓ Variance σ^2 is known
  - What does $l(\theta)$ look like?
  - How to solve for μ?
    - ✓ Let $\nabla_\theta \log L(\theta) = 0$



Likelihood Function

Log-Likelihood Function

Maximizing the Likelihood from Data

- **Maximizing Likelihood: Applied to Bayesian Models**

$$P_\theta(\mathbf{x}) = \prod_{i=1}^{n} p(x_i | pa(x_i); \theta_i)$$

$\theta = (\theta_1, \ldots, \theta_n)$ are parameters for conditionals, $\mathcal{D} = \{x^{(1)}, \ldots, x^{(m)}\}$ is the dataset

➢ **Maximum Likelihood Estimate (MLE) of the parameter θ?**

✓ Write and decompose the Likelihood function

$$L(\theta, \mathcal{D}) = \sum_{j=1}^{m} \log P_\theta(\mathbf{x}^{(j)}) = \sum_{j=1}^{m} \sum_{i=1}^{n} \log p(x_i^{(j)} | pa(x_i)^{(j)}; \theta_i)$$

✓ Solve

$$\arg\max_\theta L(\theta, \mathcal{D}) = \arg\max_\theta \log L(\theta, \mathcal{D})$$

Maximizing the Likelihood from Data

- **Maximizing Likelihood: Applied to Autoregressive Models**

$$P_\theta(\mathbf{x}) = \prod_{i=1}^{n} p_{\text{neural}}(x_i | \mathbf{x}_{<i}; \theta_i)$$

$\theta = (\theta_1, \ldots, \theta_n)$ are parameters for conditionals, $\mathcal{D} = \{x^{(1)}, \ldots, x^{(m)}\}$ is the dataset

➢ **Maximum Likelihood Estimate (MLE) of the parameter θ?**

✓ Write and decompose the Likelihood function

$$L(\theta, \mathcal{D}) = \prod_{j=1}^{m} P_\theta(\mathbf{x}^{(j)}) = \prod_{j=1}^{m} \prod_{i=1}^{n} p_{\text{neural}}(x_i^{(j)} | \mathbf{x}_{<i}^{(j)}; \theta_i)$$

✓ Solve

$$\arg\max_\theta L(\theta, \mathcal{D}) = \arg\max_\theta \log L(\theta, \mathcal{D})$$

No longer have closed-form solutions!

$$\nabla_\theta \log L(\theta) = 0$$

Maximizing the Likelihood from Data

- **Maximizing Likelihood: Applied to Autoregressive Models**
  - ➢ We would like to solve:

$$\arg\max_\theta L(\theta, \mathcal{D}) = \arg\max_\theta \log L(\theta, \mathcal{D})$$

$$\ell(\theta) = \log L(\theta, \mathcal{D}) = \sum_{j=1}^{m} \sum_{i=1}^{n} \log p_{\text{neural}}(x_i^{(j)} | \mathbf{x}_{<i}^{(j)}; \theta_i)$$

  - ➢ Maximize $l(\theta)$ via **Gradient Descent**
    - ① Initialize $\theta^0 = (\theta_1, \ldots, \theta_n)$ at random
    - ② Compute $\nabla_\theta l(\theta)$ by Back-propagation algorithm
    - ③ Update parameter $\theta^{t+1} = \theta^t + \alpha_t \nabla_\theta l(\theta)$

Maximizing the Likelihood from Data

- **Maximizing Likelihood: Applied to Autoregressive Models**

  - Maximize $l(\theta)$ via **Gradient Descent**

    ① Initialize $\theta^0 = (\theta_1, \dots, \theta_n)$ at random

    ② Compute $\nabla_\theta l(\theta)$ by Back-propagation algorithm

    ③ Update parameter $\theta^{t+1} = \theta^t + \alpha_t \nabla_\theta l(\theta)$

$$\nabla_\theta \ell(\theta) = \sum_{j=1}^{m} \sum_{i=1}^{n} \nabla_\theta \log p_{\text{neural}}(x_i^{(j)} | \mathbf{x}_{<i}^{(j)}; \theta_i)$$

  - **Question:**

    ✓ What if $m = |\mathcal{D}|$ is huge?

Maximizing the Likelihood from Data

- **Maximizing Likelihood: Applied to Autoregressive Models**

  - What if $m = |\mathcal{D}|$ is huge?

$$\begin{aligned}\nabla_\theta \ell(\theta) &= m \sum_{j=1}^{m} \frac{1}{m} \sum_{i=1}^{n} \nabla_\theta \log p_{\text{neural}}(x_i^{(j)} | \mathbf{x}_{<i}^{(j)}; \theta_i) \\ &= m E_{x^{(j)} \sim \mathcal{D}} \left[\sum_{i=1}^{n} \nabla_\theta \log p_{\text{neural}}(x_i^{(j)} | \mathbf{x}_{<i}^{(j)}; \theta_i) \right]\end{aligned}$$

  - Apply Monte Carlo Estimation to $\mathbb{E}_{x \sim \mathcal{D}}$

    ① Sample $x^{(j)} \sim \mathcal{D}$ (sample $x \sim P_{\text{data}}$ in previous sections)

    ② Compute $\nabla_\theta \ell(\theta) \approx m \sum_{i=1}^{n} \nabla_\theta \log p_{\text{neural}}(x_i^{(j)} | \mathbf{x}_{<i}^{(j)}; \theta_i)$

  - Batch Gradient Descent ➔ **Monte Carlo** ➔ Mini-Batch / SGD (**Unbiased**)



Lecture Outline

- **Learning via Maximum Likelihood**
- **Maximum Likelihood from Data**
- **Statistical Issues and the Bias-Variance Tradeoff**

Statistical Issues and the Bias-Variance Tradeoff

- **Monte Carlo Estimation**

$$\mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}[\log P_\theta(\mathbf{x})] \Rightarrow \mathbf{E}_{\mathcal{D}}[\log P_\theta(\mathbf{x})] = \frac{1}{|\mathcal{D}|}\sum_{\mathbf{x}\in\mathcal{D}} \log P_\theta(\mathbf{x})$$

  - Define loss function $\mathcal{L}(x;\theta) = -\log P_\theta(x)$

$$R(\theta) = E_{x\sim P_{data}}[\mathcal{L}(x;\theta)] \Rightarrow R_{emp}(\theta) = \frac{1}{|\mathcal{D}|}\sum_{\mathbf{x}\in\mathcal{D}} \mathcal{L}(x;\theta)$$

    - ✓ **Classification**

$$\mathcal{L}(x;\theta) = L\big(y, f(x;\theta)\big) = \begin{cases} 0 & if\ y = f(x;\theta) \\ 1 & if\ y \neq f(x;\theta) \end{cases}$$

    - ✓ **Regression**

$$\mathcal{L}(x;\theta) = L\big(y, f(x;\theta)\big) = \big(y - f(x;\theta)\big)^2$$

Statistical Issues and the Bias-Variance Tradeoff

- **Monte Carlo Estimation**

$$\mathbf{E}_{\mathbf{x}\sim P_{\text{data}}}[\log P_\theta(\mathbf{x})] \Rightarrow \mathbf{E}_{\mathcal{D}}[\log P_\theta(\mathbf{x})] = \frac{1}{|\mathcal{D}|}\sum_{\mathbf{x}\in\mathcal{D}} \log P_\theta(\mathbf{x})$$

  - Define loss function $\mathcal{L}(x;\theta) = -\log P_\theta(x)$

$$R(\theta) = E_{x\sim P_{data}}[\mathcal{L}(x;\theta)] \Rightarrow R_{emp}(\theta) = \frac{1}{|\mathcal{D}|}\sum_{\mathbf{x}\in\mathcal{D}} \mathcal{L}(x;\theta)$$

Expected / Population Risk **Empirical Risk**

  - VC Generalization Bound

$$R(\theta) \le R_{emp}(\theta) + \Phi\left(\frac{h}{l}\right)$$

h: Model Capacity

l: Amount of Data

Generalization Gap



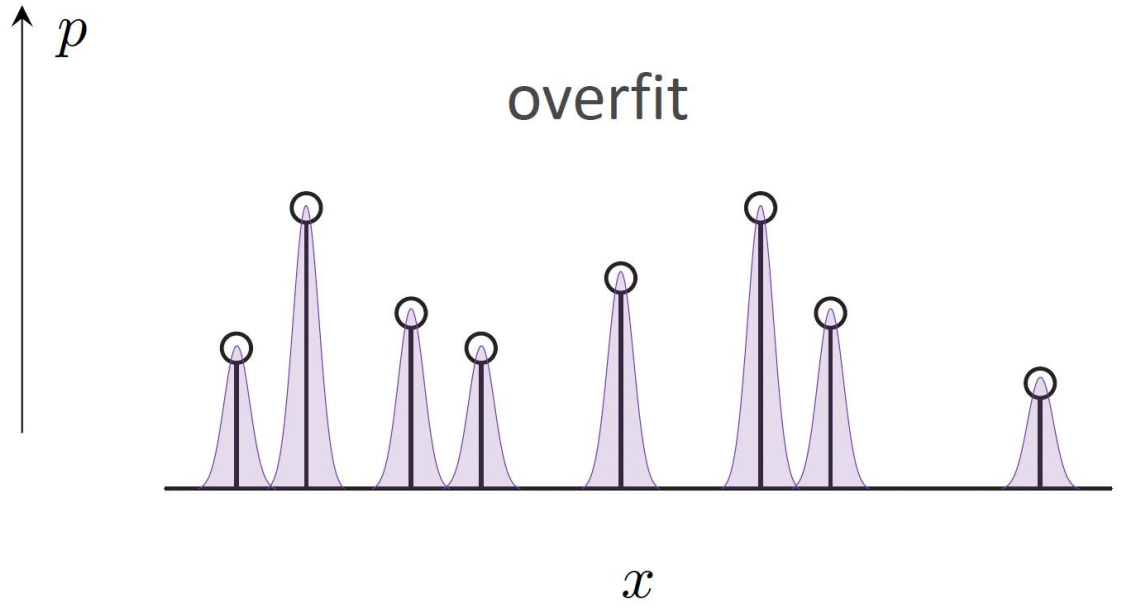
Statistical Issues and the Bias-Variance Tradeoff

- **Bias-Variance Tradeoff**
  - **Note:** Minimizing the empirical risk can easily **overfit** the data
    - ✓ Extreme case: simply memorizing the dataset $\mathcal{D}$
  - **Goal:** Make P_θ works good on samples not in $\mathcal{D}$
    - ➔ P_θ having a good **generalization ability**

How to achieve it?

Statistical Issues and the Bias-Variance Tradeoff

- **Bias-Variance Tradeoff**
  - **Overfitting:** P_θ is more than capable of representing P_{data}
    - ✓ Small perturbations on $\mathcal{D}$ will result in very different estimates



  - This type of limitation if called "**Variance**" → low generalization ability

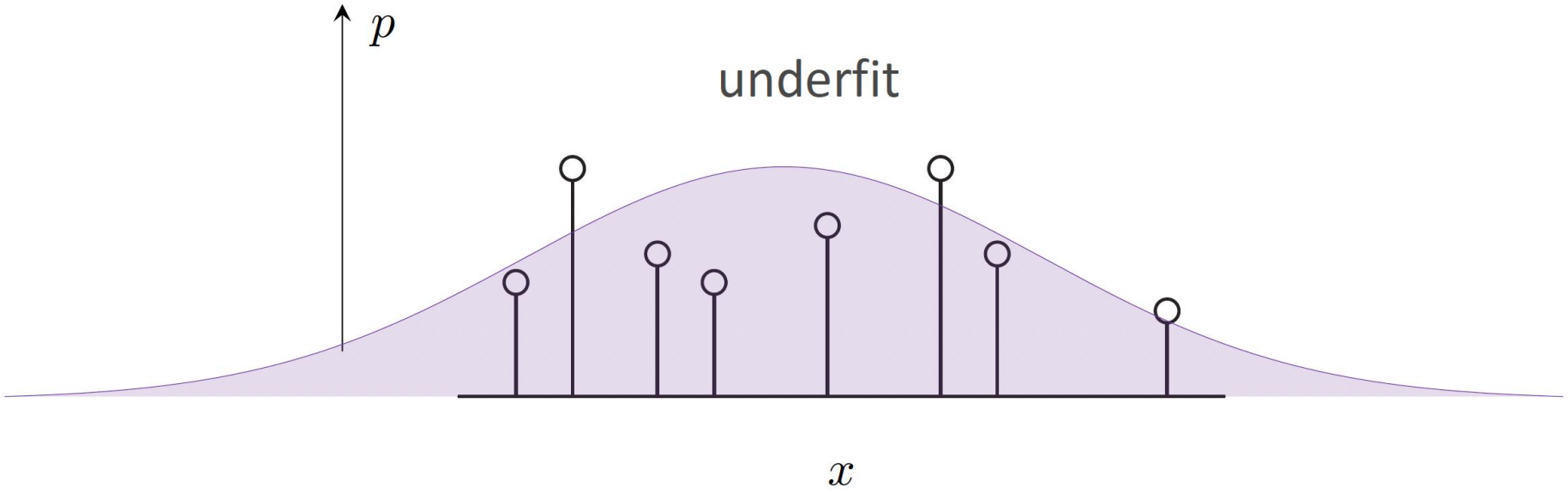
Statistical Issues and the Bias-Variance Tradeoff

- **Bias-Variance Tradeoff**
  - **Note:** Minimizing the empirical risk can easily **overfit** the data
    - ✓ Extreme case: simply memorizing the dataset $\mathcal{D}$
  - **Goal:** Make P_θ works good on samples not in $\mathcal{D}$
    - ➔ P_θ having a good **generalization ability**

    How to achieve it?
  - **Solution**
    - ① Increase the scale of dataset $\mathcal{D}$
    - ② Reduce the model capacity (**restrict the hypothesis space**)
      - ✓ Less restricted ➔ Over-fit (high variance)
      - ✓ Overly restricted?

Statistical Issues and the Bias-Variance Tradeoff

- **Bias-Variance Tradeoff**
  - **Underfitting:** P_θ is incapable to represent P_{data} , even with unlimited data
    - ✓ Can always be solved by selecting a highly expressive P_θ



  - This type of limitation is called "**Bias**" → limited hypothesis space

Statistical Issues and the Bias-Variance Tradeoff

- **Bias-Variance Tradeoff**
  - **Note:** Minimizing the empirical risk can easily **overfit** the data
    - ✓ Extreme case: simply memorizing the dataset $\mathcal{D}$
  - **Goal:** Make P_θ works good on samples not in $\mathcal{D}$

    ➔ P_θ having a good **generalization ability**

    How to achieve it?
  - **Solution**
    - ① Increase the scale of dataset $\mathcal{D}$
    - ② Reduce the model capacity (**restrict the hypothesis space**)
      - ✓ Less restricted ➔ Over-fit (high variance)
      - ✓ Overly restricted ➔ Under-fit (high bias)

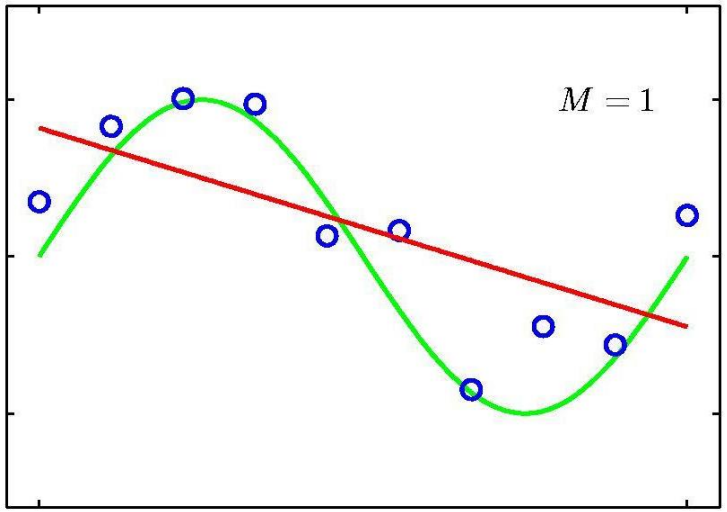
      Bias-Variance Tradeoff

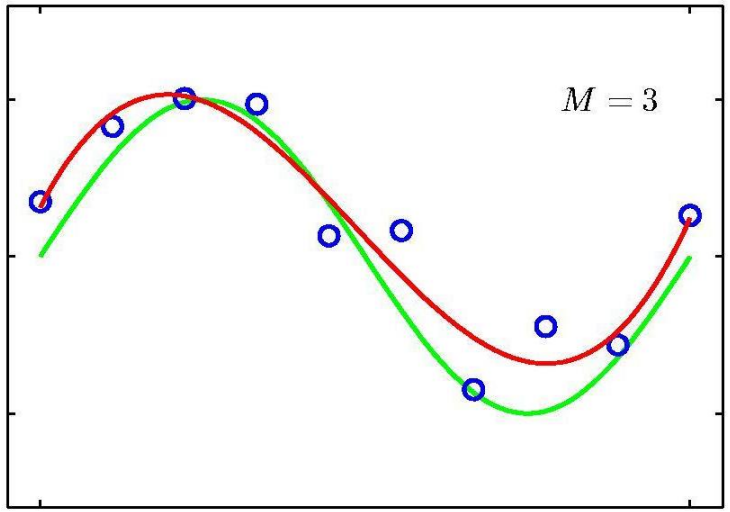
Statistical Issues and the Bias-Variance Tradeoff

- **Bias-Variance Tradeoff**

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

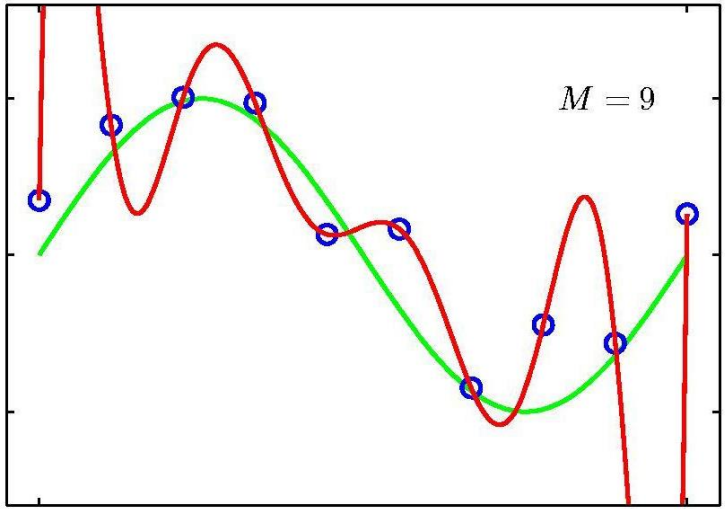
Under-Fitting

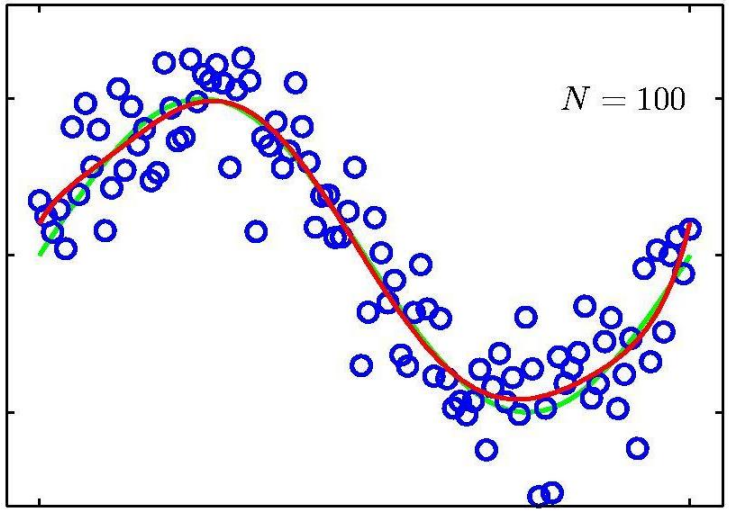




Good-Fitting

Over-Fitting



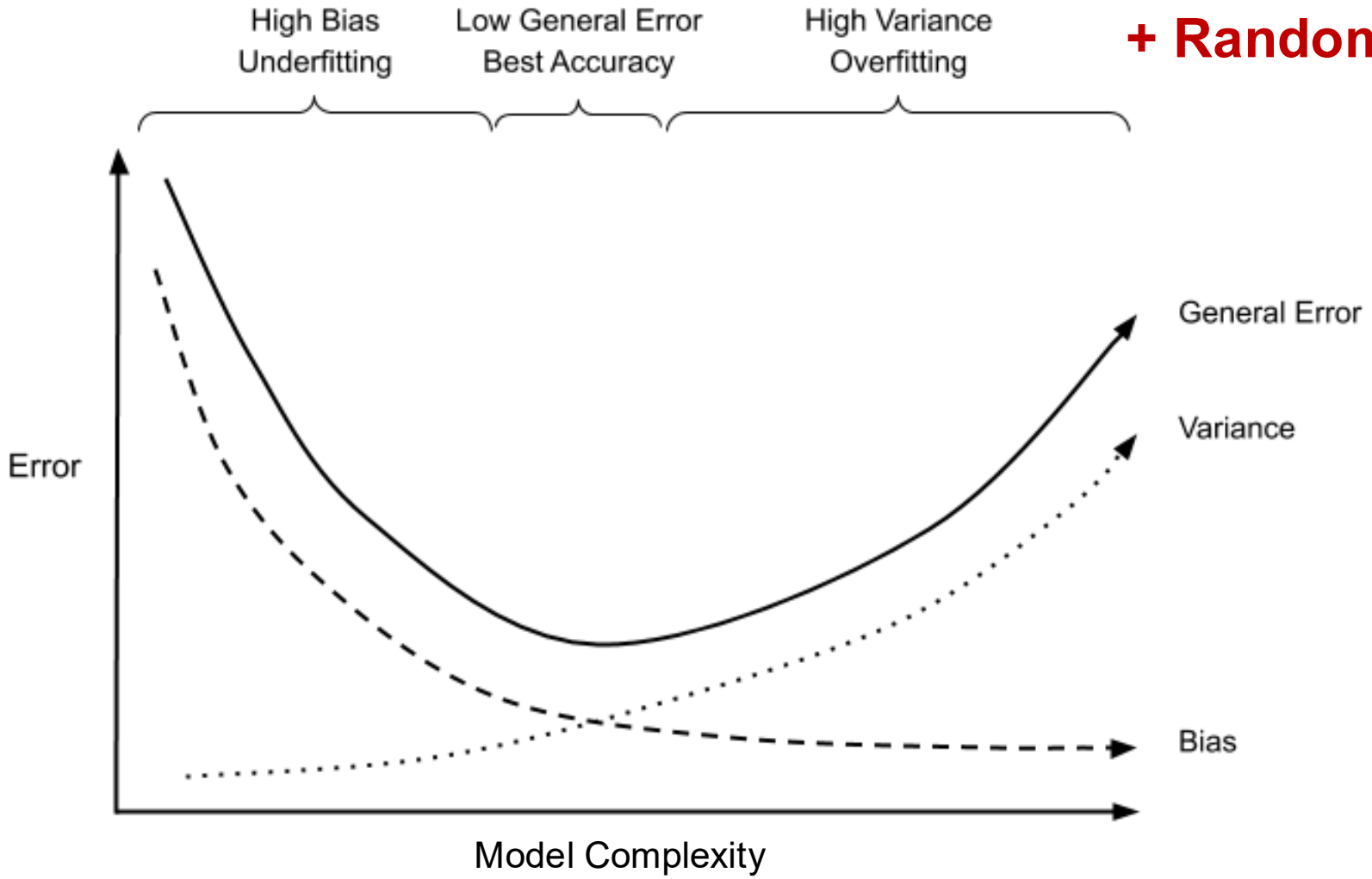


Good-Fitting
(more data)

Statistical Issues and the Bias-Variance Tradeoff

- **Bias-Variance Tradeoff**

Error = Bias + Variance + Random Error



Summary

- **Learning via Maximum Likelihood**
  - Kullback-Leibler (KL) divergence
  - $\min D_{KL}(q \parallel p) \iff \mathbf{E}_{\mathbf{x} \sim P_{\text{data}}} [\log P_\theta(\mathbf{x})]$
- **Maximizing the Likelihood from Data**
  - Monte Carlo Estimation
  - Maximum Likelihood & Gradient Descent
- **Statistical Issues and the Bias-Variance Tradeoff**
  - Expected Risk vs. Empirical Risk
  - Underfitting vs. OverFitting

Supplementary Material

- **Stanford CS 236 Lecture 4**
  - https://www.youtube.com/watch?v=bt3dqcbMLa0&list=PLoROMvodv4rPOWA-omMM6STXaWW4FvJT8&index=4
- **Cornell CS 6785 Lecture 4**
  - https://www.youtube.com/watch?v=-OjBWNmXYyg&list=PL2UML_KCiC0UPzjW9BjO-lW6dqliu9O4B&index=4
- **Deep Generative Models**
  - **Section B (Appendix)**